Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ITMO University

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

По дисциплине: Сборка образа: Dockerfile, оптимизация, healthcheck и теги

Тема работы: Знакомство с экосистемой Docker

Обучающийся: Матвеев Даниил Алексеевич

Номер ИСУ: 466659

Факультет: Факультет прикладной информатики

Группа: K3321

Направление подготовки: 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа: Программирование в инфокоммуникационных системах

Обучающийся _____ Матвеев Д. А.

дата, подпись

Руководитель _____ Аминов Н. С.

дата, подпись

Санкт-Петербург

2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **3**

1 Ход выполнения **4**

1.1 Первый рабочий образ 4

1.2 Оптимизация образа 6

1.3 .dockerignore - чистота в образе 8

1.4 Healthcheck - готовность к оркестрации 10

1.5 Финальный артефакт и тегирование 11

ЗАКЛЮЧЕНИЕ **12**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **13**

ВВЕДЕНИЕ

Сформулируйте цель и задачи лабораторной работы. Перечислите используемые инструменты; персональные параметры укажите в таблице 1, изменив набор строк под своё задание.

Таблица 1 — Персональные параметры

Параметр	Значение
Номер ИСУ	466659
Персональный порт	5489
Персональный префикс	matveev
Окружение	linux/x86_64
Секретный ключ	—

Результат хранится в ветке `lab-2` своего форка: https://github.com/DaniilMat/course-containerization-orchestration-application/tree/lab-2.

1 Ход выполнения

1.1 Первый рабочий образ

На рисунке 1 сделал Fork репозитория курса на GitHub и клонировал свой форк. Положил приложение в solutions/assist-py (без истории git оригинала).

```
MDA@Danisimo:~$ git clone https://github.com/DaniilMat/course-containerization-orchestration-application
Cloning into 'course-containerization-orchestration-application'...
remote: Enumerating objects: 178, done.
remote: Counting objects: 100% (178/178), done.
remote: Compressing objects: 100% (136/136), done.
remote: Total 178 (delta 52), reused 142 (delta 33), pack-reused 0 (from 0)
Receiving objects: 100% (178/178), 112.66 KiB | 35.00 KiB/s, done.
Resolving deltas: 100% (52/52), done.
MDA@Danisimo:~$ cd course-containerization-orchestration-application
MDA@Danisimo:~/course-containerization-orchestration-application$ git checkout -b lab-2
Switched to a new branch 'lab-2'
MDA@Danisimo:~/course-containerization-orchestration-application$ git clone https://github.com/FilBTi/assist-py solution
s/assist-py
Cloning into 'solutions/assist-py'...
remote: Enumerating objects: 29, done.
remote: Counting objects: 100% (29/29), done.
remote: Compressing objects: 100% (21/21), done.
remote: Total 29 (delta 7), reused 25 (delta 5), pack-reused 0 (from 0)
Receiving objects: 100% (29/29), 19.36 KiB | 26.00 KiB/s, done.
Resolving deltas: 100% (7/7), done.
MDA@Danisimo:~/course-containerization-orchestration-application$ rm -rf solutions/assist-py/.git
MDA@Danisimo:~/course-containerization-orchestration-application$ cd solutions/assist-py
```

Рисунок 1 — Результат этапа

Создал файл Dockerfile в корне проекта на рисунке 2.

```
  GNU nano 8.4
# 1. Базовый образ Python
FROM python:3.11-slim

# 2. Метаданные
LABEL maintainer="matveev@lab.local"
LABEL version="1.0"

# 3. Рабочая директория
WORKDIR /app

# 4. Порт приложения
ENV PORT=5489

# 5. Копируем зависимости
COPY requirements.txt .

# 6. Устанавливаем зависимости
RUN pip install -r requirements.txt

# 7. Копируем исходный код
COPY . .

# 8. Указываем порт приложения
EXPOSE ${PORT}

# 9. Запускаем FastAPI через Uvicorn
CMD uvicorn app.main:app --host 0.0.0.0 --port ${PORT}
```

Рисунок 2 — Dockerfile

Собрал образ на рисунке 3.

```
MDA@Danisimo:~/course-containerization-orchestration-application/solutions/assist-py$ docker build -t matveev-assistant:v1
.
[+] Building 224.6s (11/11) FINISHED                                                                    docker:default
 => [internal] load build definition from Dockerfile                                                             0.0s
 => => transferring dockerfile: 683B                                                                             0.0s
 => [internal] load metadata for docker.io/library/python:3.11-slim                                              2.6s
 => [auth] library/python:pull token for registry-1.docker.io                                                    0.0s
 => [internal] load .dockerignore                                                                                0.0s
 => => transferring context: 2B                                                                                  0.0s
 => [1/5] FROM docker.io/library/python:3.11-slim@sha256:07d4824036af339fa1b6ef769cc99d9c10a141769538b52b3b597c84  149.7s
 => => resolve docker.io/library/python:3.11-slim@sha256:07d4824036af339fa1b6ef769cc99d9c10a141769538b52b3b597c84f7  0.0s
 => => sha256:53daecf7856e9b7235583f1ec326387aae7d29af3d5cd2efc20246259ab817cc 1.29MB / 1.29MB                   1.9s
 => => sha256:7efbce397249e9202b070b8e638baff71160948c640b9a44f2226419a71f410d 14.46MB / 14.46MB               136.3s
 => => sha256:07d4824036af339fa1b6ef769cc99d9c10a141769538b52b3b597c84f7a7b535 10.37kB / 10.37kB                 0.0s
 => => sha256:b701198cb708b68d9413ca67d6311bd97c9389a2c5fe4da0511fc652642366cd 1.75kB / 1.75kB                   0.0s
 => => sha256:896edfbcbf0c29ad15681617a8886cb9871b7f24c5fde460611 4c8ee6bfe4c70 5.49kB / 5.49kB                  0.0s
 => => sha256:37fb86c2a767b88d500c6872195f88078f64765fe0d2f9cc375cc2efbd4c0a41 29.86MB / 29.86MB                147.4s
 => => sha256:bbe7053e6def9842d628414346e26f25e499463adb6ca48c0c8932e036e44000 249B / 249B                       2.1s
 => => extracting sha256:37fb86c2a767b88d500c6872195f88078f64765fe0d2f9cc375cc2efbd4c0a41                        1.1s
 => => extracting sha256:53daecf7856e9b7235583f1ec326387aae7d29af3d5cd2efc20246259ab817cc                        0.1s
 => => extracting sha256:7efbce397249e9202b070b8e638baff71160948c640b9a44f2226419a71f410d                        0.8s
 => => extracting sha256:bbe7053e6def9842d628414346e26f25e499463adb6ca48c0c8932e036e44000                        0.0s
 => [internal] load build context                                                                                0.0s
 => => transferring context: 281B                                                                                0.0s
 => [2/5] WORKDIR /app                                                                                           0.3s
 => [3/5] COPY requirements.txt .                                                                                0.0s
 => [4/5] RUN pip install -r requirements.txt                                                                   71.5s
 => [5/5] COPY . .                                                                                               0.0s
 => exporting to image                                                                                           0.3s
 => => exporting layers                                                                                          0.3s
 => => writing image sha256:37cecd1fac03e790de758c27671177c6b124b7d2cf1843446d3d7c3b0db47fa2                     0.0s
 => => naming to docker.io/library/matveev-assistant:v1                                                          0.0s
```

Рисунок 3 — Сборка

Запустил контейнер с переменными окружения на рисунке 4.

```
MDA@Danisimo:~/course-containerization-orchestration-application/solutions/assist-py$ docker run -d \
--name matveev-assistant-1 \
-p 5489:5489 \
-e APP_OWNER=matveev \
-e APP_SECRET=[redacted] \
matveev-assistant:v1
80c9adbd8705ee07e6a2af98e7c9890f3de1cada7344c55cd388f536751797c9
```

Рисунок 4 — Запуск

Проверил работоспособность на рисунках 5- 9.

```
MDA@Danisimo:~/course-containerization-orchestration-application/solutions/assist-py$ curl http://localhost:5489/
{"message":"👋 Привет, matveev!","service":"personal-assistant","version":"1.0","endpoints":["/me","/notes","/weather/{city}","/secret/{key}"]}MDA@Danisimo:~/course-containerization-orchestration-application/solutions/assist-py$
```

Рисунок 5 — Проверка

```
MDA@Danisimo:~/course-containerization-orchestration-application/solutions/assist-py$ curl http://localhost:5489/me
{"owner":"matveev","secret_prefix":"secret"}MDA@Danisimo:~/course-containerization-orchestration-application/solutions/assist-py$
```

Рисунок 6 — Проверка

```
MDA@Danisimo:~/course-containerization-orchestration-application/solutions/assist-py$ curl -X POST http://localhost:5489/notes \
-H "Content-Type: application/json" \
-d '{"text": "ЛР2: сборка образа"}'
{"status":"saved","note_id":1,"total":2}MDA@Danisimo:~/course-containerization-orchestration-application/solutions/assist-py$
```

Рисунок 7 — Проверка

```
MDA@Danisimo:~/course-containerization-orchestration-application/solutions/assist-py$ curl http://localhost:5489/notes
{"notes":["ЛР2: сборка образа","ЛР2: сборка образа"],"count":2}MDA@Danisimo:~/course-containerization-orchestration-appl
```

Рисунок 8 — Проверка

```
MDA@Danisimo:~/course-containerization-orchestration-application/solutions/assist-py$ curl http://localhost:5489/secret/[redacted]
{"flag":"LAB2_SUCCESS_MATVEEV","message":"The secret key is correct!"}MDA@Danisimo:~/course-containerization-orchestration-application/solutions/assist-py$
```

Рисунок 9 — Проверка

1.2 Оптимизация образа

Создан новый файл Dockerfile.optimized изображенный на рисунке 10.

В отличие от Dockerfile из этапа 1, где образ собирался в один этап, новая версия использует multi-stage сборку: зависимости устанавливаются в промежуточном образе builder, а в финальный образ runtime переносятся

только готовые пакеты и код приложения. За счёт этого в итоговый образ не попадают артефакты установки.

Установка зависимостей выполняется командой pip install –no-cache-dir –prefix=/install -r requirements.txt. Флаг –no-cache-dir отключает сохранение кэша pip, поэтому скачанные пакеты не увеличивают размер слоя. Параметр –prefix=/install позволяет установить пакеты в отдельный каталог, который затем целиком копируется в финальный образ командой COPY –from=builder /install /usr/local.

```
# Этап 1. Builder — установка зависимостей
FROM python:3.11-slim AS builder

WORKDIR /app

# Копирование списка зависимостей
COPY requirements.txt .

# --no-cache-dir: pip не хранит кэш скачанных пакетов в образе
# --prefix: ставит в отдельный каталог, чтобы целиком перенести его в runtime
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt


# Этап 2. Runtime — финальный образ
FROM python:3.11-slim AS runtime

# Метаданные
LABEL maintainer="matveev@lab.local"
LABEL version="2.0"

# Рабочая директория
WORKDIR /app

# Порт приложения
ENV PORT=5489

# Перенос только готовых пакетов, без pip-кэша и инструментов сборки
COPY --from=builder /install /usr/local

# Добавление установленных библиотек в PATH
ENV PATH="/opt/venv/bin:$PATH"

# Копирование исходного кода приложения
COPY app ./app

# Открытие порта
EXPOSE ${PORT}

# Запуск FastAPI
CMD uvicorn app.main:app --host 0.0.0.0 --port ${PORT}
```

Рисунок 10 — Dockerfile.optimized

Размеры образов изображены на рисунке 11. Размер образа v1 составил 188 МБ, образа v2 составил 164 МБ, то есть образ уменьшился на 24 МБ. Основной вес образа определяется базовым образом python:3.11-slim и самими зависимостями, поэтому разница небольшая, multi-stage сборка убирает только артефакты установки, но не сами пакеты.



Рисунок 11 — Сравнение

Ответ на вопрос о кэшировании. Копирование requirements.txt отдельно от кода ускоряет повторные сборки, потому что Docker кэширует каждый слой и сбрасывает кэш только тогда, когда изменились инструкция или копируемые ею файлы. При изменении бизнес-логики файл requirements.txt остаётся прежним, поэтому слои с копированием списка зависимостей и установкой пакетов берутся из кэша, а пересобирается только слой с кодом приложения. Если бы весь проект копировался до установки зависимостей, любое изменение кода сбрасывало бы кэш, и пакеты скачивались бы заново при каждой сборке.

1.3 .dockerignore - чистота в образе

В корне проекта создан файл .dockerignore с содержимым изображенным на рисунке 12. Также был пересобран образ.



Рисунок 12 — .dockerignore файл

Без .dockerignore в образ могли попасть все файлы из контекста сборки, которые затрагивает инструкция COPY. В этом случае это:

1. каталоги `__pycache__/` и файлы *.pyc, *.pyo (скомпилированный байт-код, который бесполезен в образе и создаётся под конкретную среду разработки);
2. каталог .git со всей историей репозитория;
3. .gitignore, README.md, Dockerfile* (служебные файлы, которые приложению не нужны);
4. виртуальные окружения .venv/ и venv/ (могут весить сотни МБ и содержать пакеты, собранные под другую систему);
5. файл .env с паролями, токенами и секретными ключами;

6. лог-файлы *.log, в которых могут быть адреса, данные пользователей и внутренняя информация.

Всё это увеличивает размер образа и замедляет сборку, потому что весь контекст передаётся демону Docker, а любое изменение таких файлов сбрасывает кэш слоя с COPY.

Наличие .git в образе является потенциальной уязвимостью, потому что в этом каталоге хранится полная история проекта. Любой, кто получил доступ к образу (например, из публичного реестра) или к контейнеру, может выполнить git log и git show, восстановить исходный код и увидеть все прошлые версии файлов. Даже если пароль, ключ или токен был удалён из кода одним из последующих коммитов, он остаётся в истории и его можно достать. Также в .git видны адреса удалённых репозиториев, имена и почты авторов, а иногда и данные для доступа в конфигурации. Помимо риска утечки, .git увеличивает размер образа.

1.4 Healthcheck - готовность к оркестрации

Добавлена проверка здоровья, в конец Dockerfile.optimized (после EXPOSE) добавлен код изображенный на рисунке 13.

```
RUN apt-get update \
    && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*

HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
CMD curl -f http://localhost:${PORT}/health || exit 1
```

Рисунок 13 — Healthcheck

Собрал финальную версию образа, проверка статуса здоровья изображена на рисунке 14.

```
NAMES                    STATUS
matveev-assistant-final  Up 40 seconds (healthy)
```

Рисунок 14 — Проверка

После добавления установки `curl` размер образа вырос на 14 МБ (рисунок 15). Рост связан с тем, что вместе с `curl` через `apt-get` устанавливаются его зависимости (`libcurl`, библиотеки для TLS, сжатия и др.).

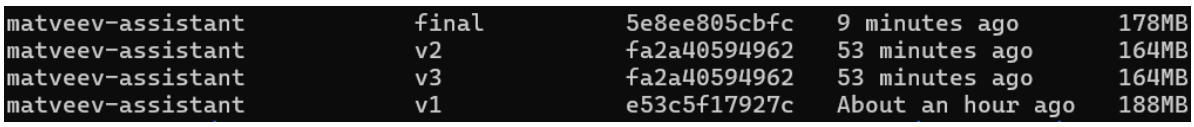


Рисунок 15 — Размеры

Проверку можно выполнить средствами самого Python, который уже есть в образе:

```
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s
--retries=3 \
CMD python -c "import os,urllib.request as u; \
u.urlopen('http://localhost:'+os.environ['PORT']+'/health')" ||
exit 1
```

Это не требует установки дополнительных пакетов и не увеличивает размер образа.

1.5 Финальный артефакт и тегирование

Добавил семантические теги. Артефакт выполнения изображен на рисунке 16.

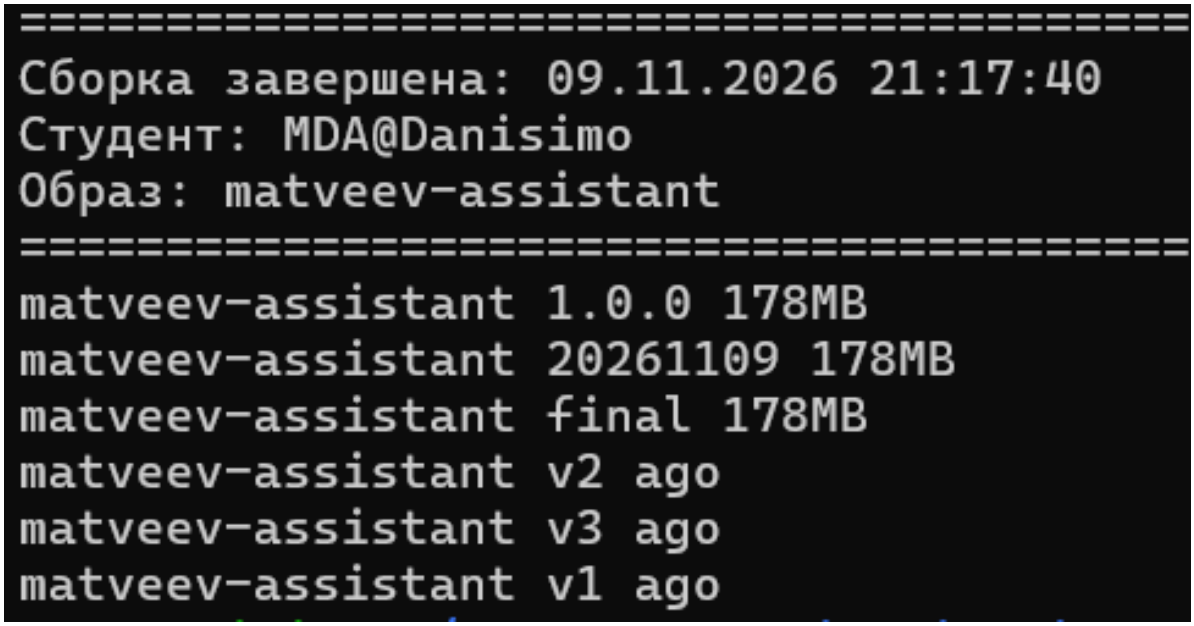


Рисунок 16 — Артефакт

ЗАКЛЮЧЕНИЕ

В ходе работы собран образ приложения на FastAPI и пройден путь от базового `Dockerfile` до образа, готового к оркестрации. Образ `matveev-assistant:v1` запущен с персональным портом 5489 и проверен запросами к эндпоинтам. В `Dockerfile.optimized` применены multi-stage сборка, отключение кэша pip и порядок слоёв, ускоряющий пересборку. Файл `.dockerignore` исключил из контекста лишние файлы. Добавленный `HEALTHCHECK` обеспечил статус `healthy`, а теги версии и даты сделали образ воспроизводимым. Полученные навыки станут основой для работ по Docker Compose и Kubernetes.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Docker Documentation: документация Docker. URL: https://docs.docker.com/ (дата обращения: 9.11.2026).